



Birla Vishvakarma Mahavidyalaya

Information Technology Department

(An Autonomous CVM Institute)

Vallabh Vidyanagar

Subject: 4IT31 (Project-I), AY: 2026-27

Project Evaluation Report

Group	09
Project Title	Chess Playing Robotic Arm
Faculty Guide	Dr. Keyur N. Brahmbhatt
Report	Progress Report - 1
Date of Evaluation	

Group Members

Student ID	Name
23IT403	Aashka Shah
23IT431	Veera Agrawal

Comment:

Faculty Guide Signature

1. System Architecture

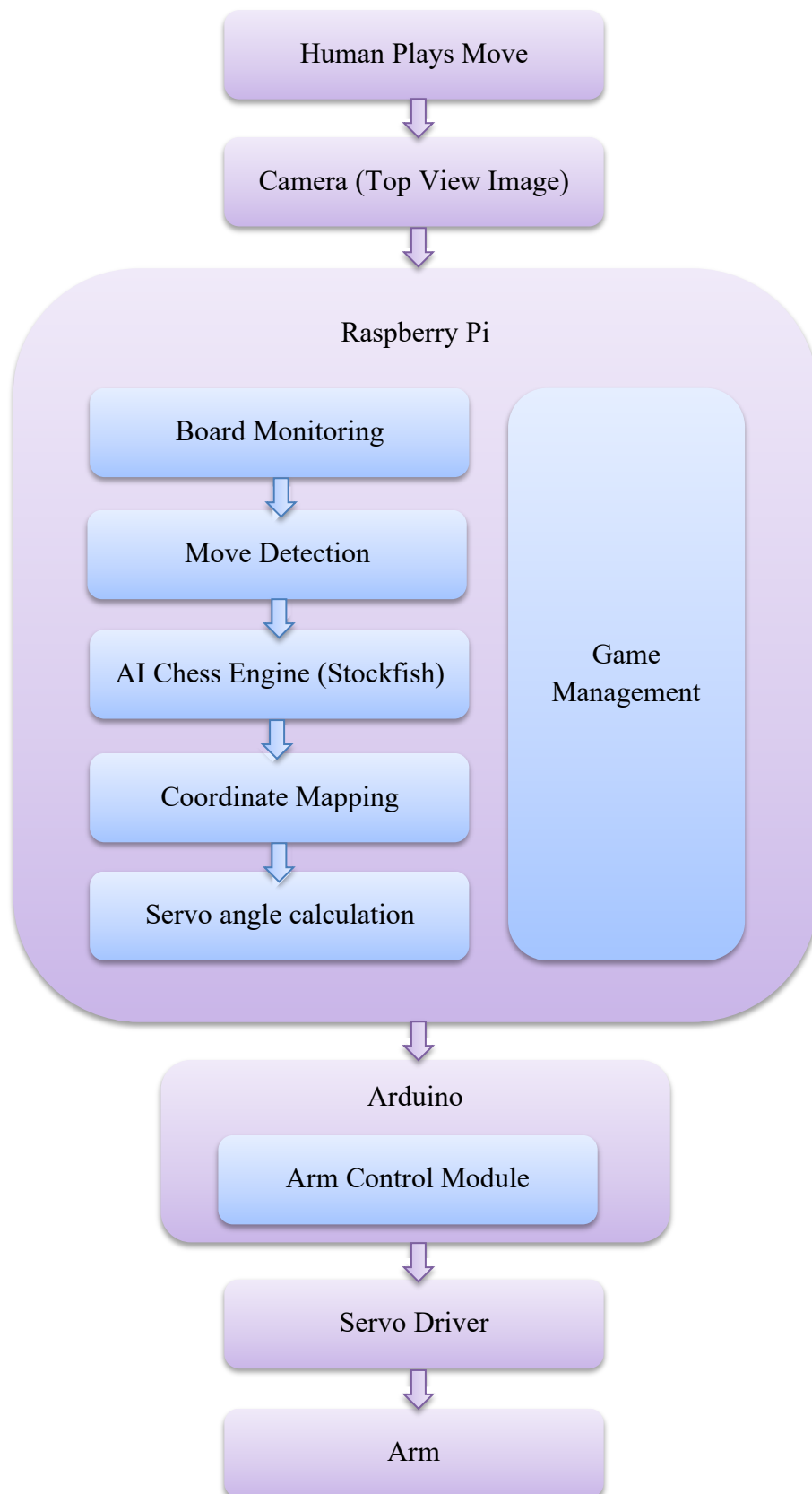


Figure 1.1 System Architecture Diagram

2. Development Methodology

The Chess Playing Robotic Arm System follows the Incremental Development Model, in which the complete system is developed and tested in multiple stages. Each increment adds new functionality while building upon the previously completed modules. This approach allows individual hardware and software components to be tested independently before integrating them into the complete system.

The development process is divided into the following phases:

2.1. Requirement Analysis

- Identify functional and non-functional requirements.
- Select hardware components such as Raspberry Pi, Arduino Uno, camera, servo motors, and robotic arm.
- Define system architecture and module interactions.

2.2. System Design

- Design the software architecture and hardware connections.
- Develop the coordinate mapping strategy and communication protocol.
- Plan the robotic arm workspace and camera placement.

2.3. Module Development

Each module is developed independently:

- Camera-Based Board Monitoring
- Move Detection Module
- Chess AI Engine (Stockfish Integration)
- Coordinate Mapping Module
- Robotic Arm Control Module
- Game Management Module

2.4. Integration

- Integrate all software modules on the Raspberry Pi.
- Establish serial communication between Raspberry Pi and Arduino.
- Connect the robotic arm and verify synchronized operation.

2.5. Testing

- Perform unit testing for individual modules.
- Conduct integration testing to verify communication between modules.
- Validate robotic arm movement accuracy and chess gameplay correctness.

3. Task Allocation

The project tasks have been carried out collaboratively, with responsibilities shared across all phases of development. The tasks were broadly categorized into software aspects, including camera-based board monitoring, move detection, chess AI integration, and game management, and the hardware aspects, including robotic arm assembly, coordinate mapping and inverse kinematics on the Raspberry Pi, Arduino-based servo control, and robotic arm operation. Both team members collaborated on requirement analysis, system integration, testing, debugging, and optimization to ensure the successful implementation of the complete system.

4. Implementation Roadmap

4.1. Phase wise roadmap

4.1.1. Requirement Analysis & Planning

Finalize project requirements, study chess rules, select hardware (Raspberry Pi, Arduino, Camera, Servo Motors), prepare SRS

4.1.2. Hardware Setup

Assemble the 6-DOF robotic arm, connect servo motors, servo driver (PCA9685), Arduino, Raspberry Pi, and power supply

4.1.3. Camera Calibration & Board Monitoring

Install camera, calibrate for top-view capture, implement chessboard detection using OpenCV, identify board corners

4.1.4. Piece Recognition & Move Detection

Detect chess pieces, generate digital board state, compare consecutive board states to identify player's move, validate moves

4.1.5. Chess AI Integration

Integrate Stockfish with Raspberry Pi, convert board state to FEN, generate AI moves based on selected difficulty

4.1.6. Coordinate Mapping & Motion Planning

Map chessboard squares (A1–H8) to robotic arm coordinates, implement inverse kinematics, generate smooth movement paths

4.1.7. Robotic Arm Control

Program Arduino for servo control, establish UART communication, execute pick-and-place operations, handle captures

4.1.8. Game Management & User Interface

Develop GUI for starting games, selecting difficulty, displaying timers and move history, manage game state

4.1.9. System Integration

Integrate all modules, synchronize camera, AI, and robotic arm, test end-to-end gameplay

4.1.10. Testing & Optimization

Perform unit, integration, and system testing, calibrate robotic arm accuracy, optimize speed and reliability

5. Work plan

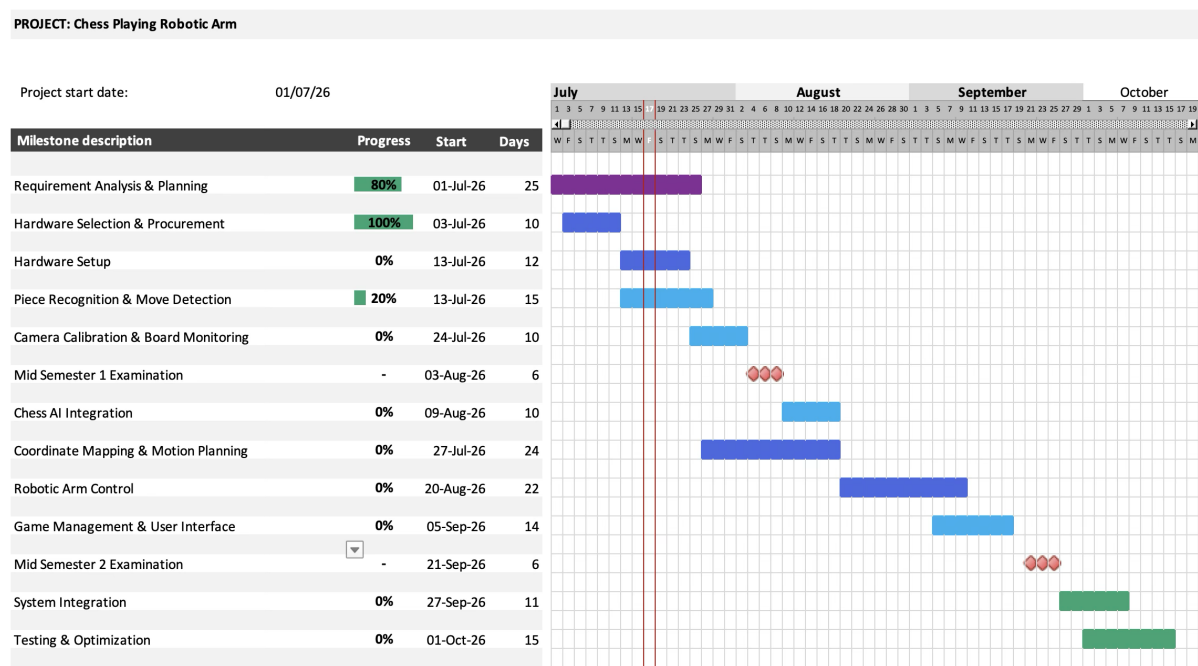


Figure 5.1 Gantt Chart